

JS Objects

1. What is an Object in Javascript?

- a. In Javascript an object is a collection of multiple different properties in the form of key-value pairs.
- b. **OR**
- c. An object is a programmatical representation of a real life entity.
- d. **Property** → It is the combination of key and value pair.
- e. Where internally all the keys (or properties) in an object stored in the form of string.
- f. When you print (console) an object, the keys might not appear in the order you added them. *Object properties do not follow a specific order.*
- g. If an object has two keys with the same name, the latest one will replace (override) the previous one.

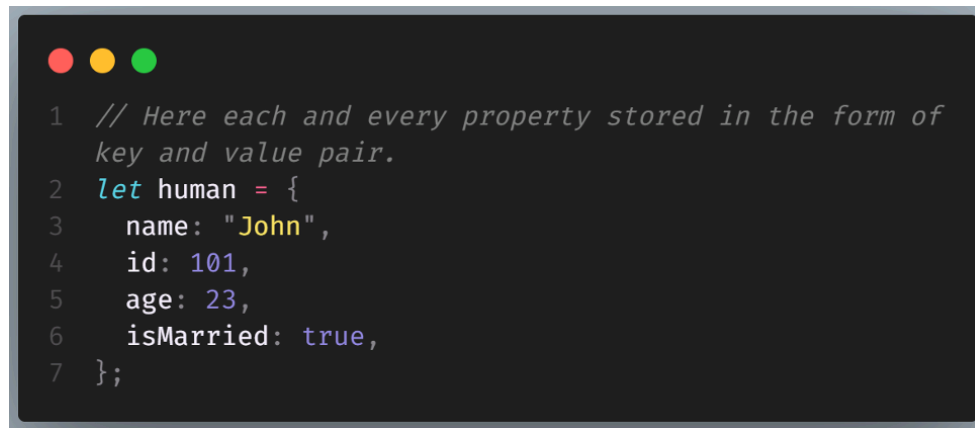
2. Why do we need to use objects in JS?

- a. Using objects in JavaScript is essential because they provide a powerful and flexible way to organize, manage, and manipulate data in your code.
- b. **For Example** → Let's consider that we have created an array which consists of values like, name, id, age, married status, etc.



```
let human = ["John", 101, 23, true];
```

- c. It's hard to know what each value represents (e.g., name, ID, age, isMarried).
- d. **Why use an object?**
- e. Objects let you store data as key-value pairs, making it easier to understand what each value represents.



```
1 // Here each and every property stored in the form of
  // key and value pair.
2 let human = {
3   name: "John",
4   id: 101,
5   age: 23,
6   isMarried: true,
7 };
```

- f. **Grouping Related Data Together** → Objects allow you to group related data (properties) and behavior (methods) into a single structure.
- g. **Representing Real-World Entities** → Objects make it easy to model real-world entities with properties and methods.

3. Syntax to Create Object:

- a. We can create the object by using object literal in the form of key-value pair:
- b. **Syntax:**

```
let obj_ref_var = {
  key1: "value1",
  key2: "value2",
  key3: "value3",
};
```

4. Accessing the Object:

a. By Using Dot (.) Notation:

- i. The dot notation is the most common way to access properties, but it doesn't work for special keys.

- ii. **For Example** →

```
let person = {  
  name: "john",  
  age: 25,  
  isMarried: true,  
  "dob-year": 1879,  
};
```

- iii. **Syntax to access object properties:**

```
console.log(person.name); //john  
console.log(person.age); //25  
console.log(person.isMarried); // true
```

b. By Using Square Bracket ([]) Notation:

- i. The bracket notation is used when:
 - 1. The key is dynamic.
 - 2. The key has special characters or spaces.
 - 3. The key consists of only numbers.
- ii. **Syntax to access object properties by using bracket notation:**

```
console.log(person["dob-year"]); //1879
```

- c. **Important Note:** You cannot access normal keys with bracket notation unless you use quotes around the key name.

```
console.log(person[name]); // Undefined  
console.log(person["name"]); //john
```

5. Accessing the array elements inside the object:

a. For Example →

```
let student = {  
  sName: "john",  
  sAge: 20,  
  sSkills: ["HTML", "CSS", "JS"],  
};  
  
console.log(student.sName); // john  
console.log(student.sAge); //20  
console.log(student.sSkills); // (3) ['HTML', 'CSS', 'JS']  
console.log(student.sSkills[0]); // HTML
```

6. Ways of Creating Object:

a. We can create the object various ways so let's see one by one:

i. By Using Object Literals: {}

1. The most common and straightforward way to create an object is using an object literal.
2. It is defined with curly braces {}.
3. **Deleting a Property from an Object in JavaScript** ⇒
4. If we want to remove or delete any property we have to use **delete** keyword.
5. JavaScript has a special keyword called "**delete**" that allows you to delete or remove any properties you wish.
6. **Syntax** → **delete** object.propertyName ;

ii. In-built Object Constructor:

1. JavaScript provides a built-in **Object()** constructor that can be used to create objects.

2. **For Example** →

```
let obj = new Object();  
console.log(obj); //{}
```

```
//! Adding the properties inside the  
object
```

```
obj.eName = "John";  
obj.eAge = 20;
```

```
console.log(obj); // {eName: 'John',  
eAge: 20}
```

iii. Functional Constructor:

1. A **functional constructor** is a function that is used to create and initialize objects when called with the **new** keyword.
2. You can define a custom constructor function to create multiple objects with the same structure.

3. **For Example** →

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

```
let user1 = new Person("john", 20);  
let user2 = new Person("smith", 25);
```

```
console.log(user1);    //Person    {name:
'john', age: 20}
console.log(user2);    //Person    {name:
'smith', age: 25}
```

7. What is a Method in JavaScript?

- a. In JavaScript, a **method** is simply a **function** that is stored as a **property of an object**.

b. **For Example** →

//Traditional Way to create the method

```
let car = {
  cName: "Audi",
  cModel: "A50",
  drive: function () {
    console.log("Car Started🚗");
  },
};
car.drive(); //Car Started🚗
```

//Shorter Method Syntax (ES6):

```
let car = {
  cName: "Audi",
  cModel: "A50",
  drive() {
    console.log("Car Started🚗");
  },
};
car.drive(); //Car Started🚗
```

8. Standard Object In-built Methods:

a. Object.keys():

- i. The **Object.keys()** method returns an array with the keys of an object.
- ii. The **Object.keys()** method does not change the original object.
- iii. **Syntax:**



```
Object.keys( obj );
```

- iv. **Return Type** → It will return an array containing the keys of the object.

b. Object.values():

- i. The **Object.values()** method returns an array of the property values of an object.
- ii. The **Object.values()** method does not change the original object.
- iii. **Syntax:**



```
Object.values( obj );
```

- iv. **Return Type** → It will return an array containing the values of the object.

c. Object.entries():

- i. The **Object.entries()** method returns each key and value pair in the form of an array.
- ii. Get an array of **[key, value]** pairs.

iii. **Syntax:**

 `Object.entries( obj );`

- iv. **Return Type:** It will return an iterable array of the object in the form of key-value pairs.

d. **Object.fromEntries():**

- i. The `Object.fromEntries()` method creates an object from a list of key-value pairs.

ii. **Syntax:**

 `Object.fromEntries( arr );`

- iii. **Return Type:** It will return object created from the key-value pairs.

e. **Object.assign():**

- i. The `Object.assign()` method copies properties from one or more source objects to a target object.
- ii. Objects are assigned and copied by reference.

iii. **Syntax:**

 `Object.assign( target_object, sources );`

- iv. **Return Type:** It will return the target object.

f. **Object.seal():**

- i. The `Object.seal()` method prevents additions or deletions of new properties.
- ii. But we can *modify* the property.

iii. **Syntax:**

 `Object.seal( obj );`

- iv. **Return Type:** It will return the object which has been sealed.

g. Object.isSealed():

- i. The `Object.isSealed()` method can be used to check if an object is sealed.
- ii. The `Object.isSealed()` returns `true` if an object is sealed.
- iii. **Syntax:**

 `Object.isSealed( obj );`

- iv. **Return Type:** It will return the boolean value as `true` if the object is sealed and `false` if not.

h. Object.freeze():

- i. The `Object.freeze()` method prevents any changes to an object.
- ii. Frozen objects are read-only.
- iii. The `Object.freeze()` method freezes an object that we can *not add new property, modify the property and delete the property*.
- iv. **Syntax:**

 `Object.freeze( obj );`

- v. **Return Type:** It will return the frozen object.

i. **Object.isFrozen():**

- i. The **Object.isFrozen()** checks if the passed object is frozen or not.

ii. **Syntax:**



```
Object.isFrozen( obj );
```

- iii. **Return Type:** It will return the boolean value **true** if the object is frozen and **false** if not.

j. **Object.hasOwn():**

- i. The **Object.hasOwn()** static method returns **true** if the specified object has the indicated property as its **own property**.
- ii. If the property is inherited, or does not exist, the method returns **false**.

iii. **Syntax:**



```
Object.hasOwn( object, "property" );
```

- iv. **Return Type:** It will return the boolean value, **true** if the specified property is its own property or **false**.